Probabilistic Motion Models: Velocity and Odometry

Contents

1. Overview . . . 2
2. Motion as a distribution . . . 3
   - 2.1 Why one command gives many end poses . . . 3
   - 2.2 The motion model and its density . . . 3
3. Wheel odometry: measuring the motion . . . 5
   - 3.1 From encoder ticks to a new pose . . . 5
   - 3.2 Why odometry drifts, and why it needs calibration . . . 7
4. The odometry motion model . . . 8
   - 4.1 Odometry or commands: which data to use . . . 8
   - 4.2 Turn, drive, turn . . . 9
   - 4.3 Noise on each of the three parts . . . 11
   - 4.4 Scoring a possible end pose . . . 12
5. Drawing sample poses . . . 13
   - 5.1 Why sample instead of scoring . . . 13
   - 5.2 Sampling the odometry model . . . 13
6. The velocity motion model . . . 15
   - 6.1 The exact arc from the commanded speeds . . . 15
   - 6.2 Noise, and why it needs a third number . . . 17
7. Ruling out poses inside walls . . . 19
8. Summary . . . 21
9. Sources . . . 21
10. Key terms . . . 22

1. Overview

> **Key point:** A command or an odometry reading does not fix where the robot ends up; it fixes a probability distribution over likely end poses. A probabilistic motion model describes that distribution, so that a filter can both score any end pose and draw likely end poses at random.

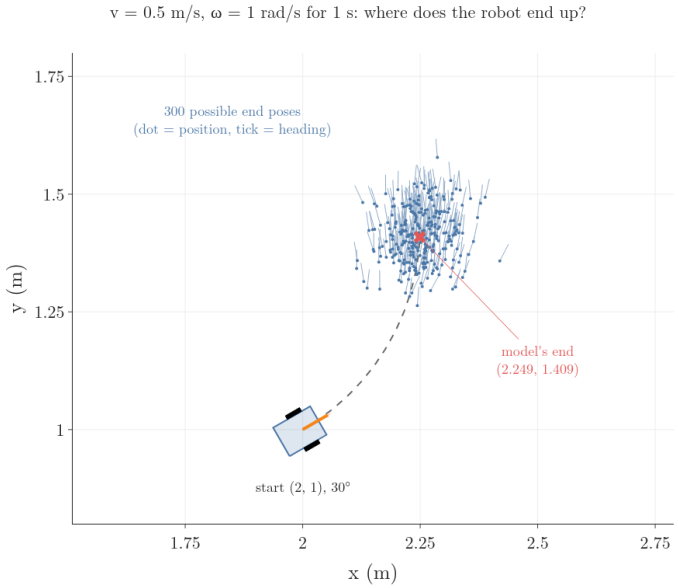


Figure 1: The robot at (2, 1) facing 30 degrees drives 0.5 m/s while turning 1 rad/s for 1 s. The kinematic model predicts one end pose, (2.249, 1.409); 300 sampled end poses form a cloud around it, each with a slightly different heading

So far the robot can predict where a command takes it, with the kinematic model (G-2297), and it knows that every move makes it less sure where it is. What still breaks: the kinematic model predicts exactly one place. Our robot from that model starts at the pose (2 m, 1 m, 30°). It drives forward at 0.5 m/s, its **linear velocity** v (G-2291), while its heading turns at 1 rad/s, its **angular velocity** ω (G-2292). After 1 s it should stand at (2.249 m, 1.409 m), facing 87.3°. A real robot on a real floor ends somewhere near there, but a little off, and a different amount off every time (Figure 1). We leave the 10-cell hallway of RO-007 for this robot on a floor, because a real move turns and drives in the plane.

This Note adds the models that say how far off the robot ends, and in which directions:

- motion as a probability distribution over end poses, and why its value is a density (Section 2);
- wheel odometry: how wheel encoders measure the motion, and why it drifts (Section 3);
- the odometry motion model: turn, drive, turn, each with its own noise (Section 4);
- drawing random end poses from a motion model (Section 5);
- the velocity motion model, for when only the commanded speeds are known (Section 6);

- using a map to rule out end poses inside walls (Section 7).

2. Motion as a distribution

> **Key point:** Wheels slip and motors lag, so the same command ends in different poses. We describe the result as a distribution over end poses, $p(x_t \mid u_t, x_{t-1})$, whose value at a pose is a density, not a probability.

2.1 Why one command gives many end poses

Why does the robot of Figure 1 end a little off, and a different amount off every time? Ask a robot to move 1 m forward and it may move 95 cm one time and 1.1 m the next. The reasons are physical:

- the motors do not reach the commanded speed at once, so the robot covers less ground than planned;
- a wheel slips on a smooth floor or a carpet, so it turns without moving the robot as far;
- a bump, such as a cable on the floor, makes a wheel skip part of a turn;
- the two wheels differ slightly in diameter, or a load presses one side harder, so the robot drifts to one side.

The last two are listed for wheeled robots in the Freiburg slides on motion models. Their effects add up from move to move: a robot that counts only its own moves through a maze slowly drifts away from its true path, the dead reckoning (G-2311) drift of the previous Note. So no rule can give the one end pose; what we can give is how likely each end pose is.

2.2 The motion model and its density

A pose has three numbers, which makes its distribution hard to draw, so we start with a robot that moves along a straight line. It starts at 0 m and is told "drive 1 m". From many trials we find that it ends near 1 m, with a standard deviation (G-1870) of 0.1 m, in a normal distribution (G-827). Figure 2 draws the curve. We can ask it questions such as "how likely is the robot to end at 1.1 m?"

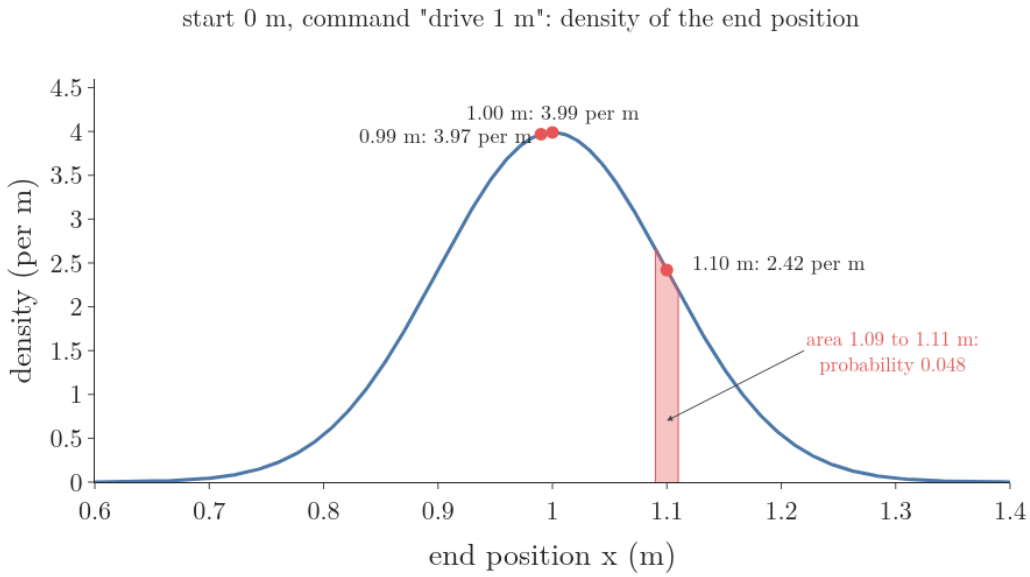


Figure 2: The density of the end position after the command "drive 1 m": a Gaussian peaking at 1 m with 3.99 per m; at 0.99 m it is 3.97 per m and at 1.10 m 2.42 per m; the area between 1.09 and 1.11 m, the probability of ending there, is 0.048

The curve's height at a point is a probability density (G-1569): probability per metre. It is not itself a probability, because the chance of ending at exactly 1.1 m, to infinitely many decimals, is zero. A probability is an area under the curve. For the 2 cm band from 1.09 m to 1.11 m:

$$\text{density at } 1.10 \text{ m} = 2.42 \text{ per m}$$

$$\text{band width} = 0.02 \text{ m}$$

$$\text{probability} \approx 2.42 \times 0.02 = 0.048$$

The exact area is 0.0484. A density can be larger than 1: at 1 m it is 3.99 per m. Some course slides call these density values "probabilities"; they are densities, and only their areas are probabilities.

The same idea in full is the **motion model** (G-2319), also called the state transition probability (Freiburg slides):

$$p(x_t \mid u_t,\ x_{t-1})$$

- x_{t-1} is the pose before the move, such as (2 m, 1 m, 30°);
- u_t is the control: the command or the odometry reading for this move;

- x_t is a possible pose after the move;
- the value is the density of ending at x_t.

In the one-dimensional example:

$$p(1.1 \mid \text{drive 1 m},\ 0) = 2.42 \text{ per m}$$

A filter uses a motion model in two ways, and this Note builds both:

1. **Score a pose:** given a possible end pose, compute its density (Section 4.4). Filters that keep a probability for every cell of a grid need this.
2. **Draw poses:** produce random end poses that follow the distribution (Sections 5 and 6). Filters that keep a cloud of sample poses, the particle filters, need this.

3. Wheel odometry: measuring the motion

> **Key point:** Encoders count how far each wheel turned; from the two wheel distances we get the robot's forward move and turn, and from those its new pose. Adding up many such steps is accurate over short distances but drifts, from slip and from wrong wheel sizes.

3.1 From encoder ticks to a new pose

The motion model needs a control u_t, the command or an odometry reading. To see how far off a move went, the robot can measure what its wheels really did instead of trusting the command. A **wheel encoder** (G-2320) is a sensor that counts how far a wheel turns. A common kind is a disc with evenly spaced slots fixed to the motor shaft. A light shines through the slots onto a light sensor, which gives one pulse, or tick, each time a slot passes (Freiburg slides). Knowing how many slots make a full turn, the robot turns the tick count into the angle the wheel turned (Figure 3).

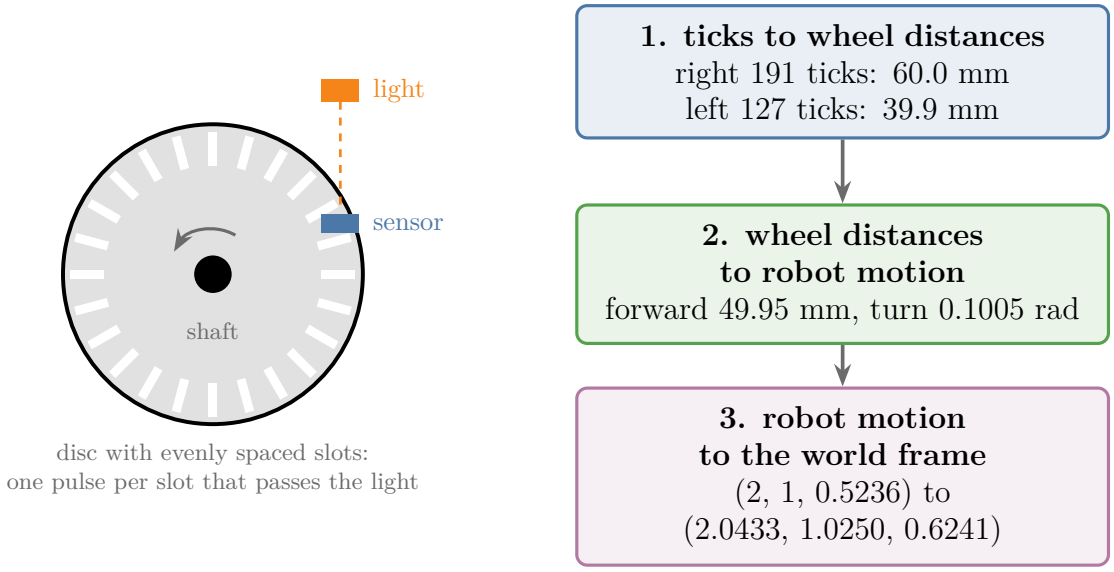


Figure 3: Left: an optical encoder, a slotted disc on the wheel shaft between a light and a sensor; each passing slot gives one tick. Right: the three steps of wheel odometry with the Note's numbers: 191 and 127 ticks give wheel distances 60.0 mm and 39.9 mm, which give a forward move of 49.95 mm and a turn of 0.1005 rad, which move the pose from (2, 1, 0.5236) to (2.0433, 1.0250, 0.6241)

Working out the robot's pose by adding up the wheel motion the encoders measure is **wheel odometry** (G-2321). It takes three steps. Our encoders give 1000 ticks per wheel turn; our wheels have radius $r = 0.05$ m and are $L = 0.2$ m apart (RO-001). In one step of 0.1 s the right encoder counts 191 ticks and the left 127.

Step 1: ticks to wheel distances. A rolling wheel moves one circumference per turn, by rolling without slipping (G-2290). One tick is a thousandth of that:

$$\text{tick} = \frac{2\pi \times 0.05}{1000}$$

$$= 0.314 \text{ mm}$$

$$d_R = 191 \times 0.314 = 60.0 \text{ mm}$$

$$d_L = 127 \times 0.314 = 39.9 \text{ mm}$$

Step 2: wheel distances to the robot's move. These are the forward kinematics (G-2294) of RO-001, used with distances instead of speeds. The time step cancels out, so the robot needs only the tick counts, not the wheel speeds:

$$\Delta s = \frac{d_R + d_L}{2}$$

$$= \frac{60.0 + 39.9}{2} = 49.95 \text{ mm}$$

$$\Delta\theta = \frac{d_R - d_L}{L}$$

$$= \frac{0.0600 - 0.0399}{0.2} = 0.1005 \text{ rad}$$

Step 3: the move in the world frame. An Euler step (G-2299) from the pose (2, 1, 0.5236):

$$x = 2 + 0.04995 \times 0.866 = 2.0433$$

$$y = 1 + 0.04995 \times 0.5 = 1.0250$$

$$\theta = 0.5236 + 0.1005 = 0.6241$$

The step repeats every 0.1 s, each one starting from the last result. The tick counts match the wheel spin rates of RO-001 (12 and 8 rad/s), so the result matches the first Euler step there. Each step adds to the last, so an error made in one step stays in every later pose.

3.2 Why odometry drifts, and why it needs calibration

Wheel odometry adds a small increment to its previous estimate again and again, so every small error stays in the sum and the estimate drifts. The errors are of two kinds:

- **Random errors**, different every time: wheel slip, bumps, the step-by-step approximation. When a wheel slips, the encoder still counts ticks but the robot does not move as far.
- **Systematic errors**, the same every time: the model's wheel radius or wheel separation is slightly wrong. No two real robots are identical, even when built to the same drawing.

Figure 4 shows how much a small systematic error costs. The model assumes both wheels have radius 0.05 m, but the right one is really 0.0505 m, 1 percent larger. Both encoders count the same ticks, so odometry believes the robot drove straight for 10 m. Really, the right wheel covered 10.1 m and the left 10 m, so the robot turned left:

$$\Delta\theta = \frac{10.1 - 10}{0.2} = 0.5 \text{ rad}$$

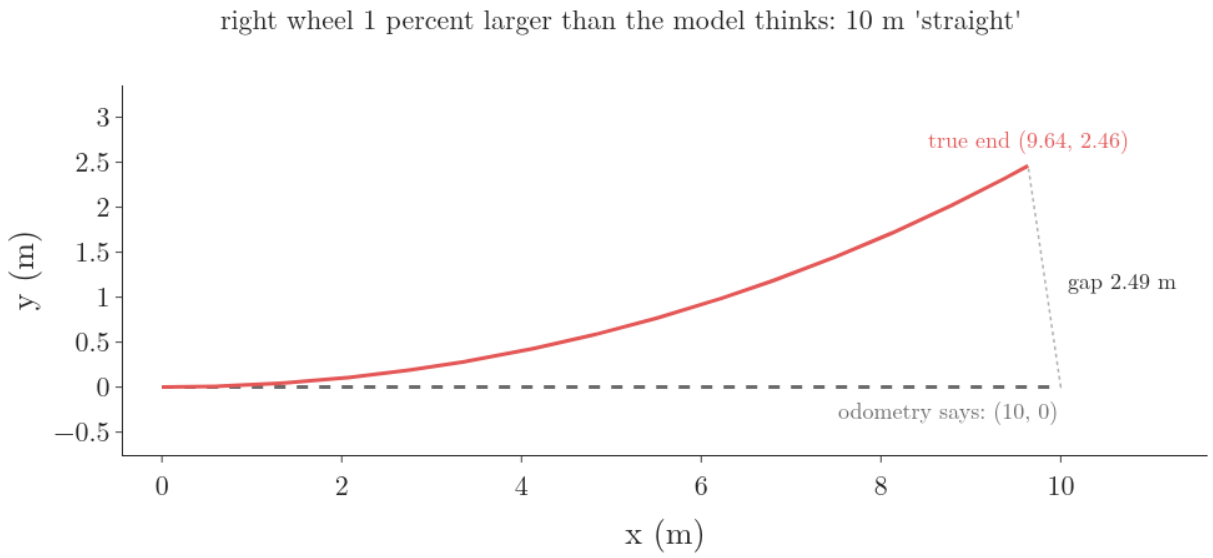


Figure 4: A robot whose right wheel is 1 percent larger than the model thinks: odometry says it drove straight to (10, 0); really it curved left to (9.64, 2.46), 2.49 m away

The true robot ends at (9.64 m, 2.46 m), 2.49 m from where odometry puts it. Finding the real wheel radii and separation of one particular robot by driving known paths is **odometry calibration** (G-2322); it removes most of the systematic error, while the random error stays and needs a probabilistic model.

4. The odometry motion model

> **Key point:** The odometry model takes the motion odometry reports between two poses, splits it into turn, drive, turn, and puts separate noise on each part. Its density at a possible end pose is the product of three normal densities.

4.1 Odometry or commands: which data to use

Now the cloud of Figure 1 itself: which end poses around (2.249, 1.409) are likely, and which are not? There are two common choices for the control u_t of a wheeled robot, and each gives its own motion model: the **odometry motion model** (G-2323), built in this section, and the **velocity motion model** (G-2324) of Section 6 (Freiburg slides):

	Odometry model	Velocity model
Data	the motion measured by wheel encoders	the commanded speeds v and ω
Needs	encoders	nothing extra
Accuracy	usually better	usually worse
Available	only after the move	before the move

Odometry is usually more accurate, because it measures what the wheels did after the command was carried out, while the velocity model only hopes the command was followed. Odometry also leaves out the error of the model that turns speeds into motion.

Strictly, odometry is a measurement. We still treat it as a control, because treating it as a measurement would mean adding the robot's speeds to the state, a bigger state for the same result. Its one drawback: it exists only after the move, so a planner that must predict motion before it happens uses the velocity model instead.

Odometry's pose estimate drifts from the truth over time, as Section 3.2 showed. So the model does not use odometry's poses themselves. It uses only the change odometry reports over one short step, which stays close to the true change even when odometry's absolute pose has drifted.

4.2 Turn, drive, turn

Odometry reports that the robot went from one pose to another. Over our 1 s drive:

$$\bar{x}_{t-1} = (2,\ 1,\ 0.524)$$

$$\bar{x}_t = (2.249,\ 1.409,\ 1.524)$$

The bars mark odometry's own estimates. Whatever path the robot took, any such change of pose can be done in three simple motions: turn on the spot to face the end point, drive straight to it, turn on the spot to the end heading (Figure 5). Three numbers describe the three motions, matching the three numbers of the pose change (Thrun et al. 2005 §5.4, as in the Freiburg slides):

- δ_{rot1}: the first turn;
- δ_{trans}: the straight drive;
- δ_{rot2}: the second turn.

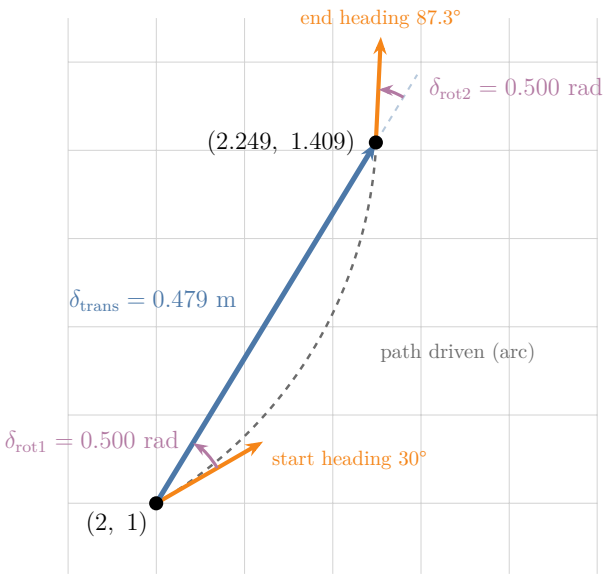


Figure 5: Odometry from (2, 1) facing 30 degrees to (2.249, 1.409) facing 87.3 degrees, as turn, drive, turn: first turn 0.500 rad to face the end point, drive 0.479 m along the straight line, second turn 0.500 rad to the end heading. The robot really drove the dashed arc

The drive is the straight-line distance between the two positions:

$$\delta_{\text{trans}} = \sqrt{0.249^2 + 0.409^2}$$

$$= 0.479 \text{ m}$$

The first turn is the direction of that line minus the start heading. The direction of a line that rises 0.409 m while running 0.249 m comes from the **atan2** (G-2325) function, which takes the rise and the run separately and returns the angle over the full circle. The plain inverse tangent of the ratio covers only half the circle, so it cannot tell "up and right" from "down and left" (Freiburg slides):

$$\text{atan2}(0.409,\ 0.249) = 1.024$$

$$\delta_{\text{rot1}} = 1.024 - 0.524 = 0.500 \text{ rad}$$

The second turn is whatever turning is left:

$$\delta_{\text{rot2}} = 1.524 - 0.524 - 0.500$$

$$= 0.500 \text{ rad}$$

4.3 Noise on each of the three parts

The true motion is odometry's motion plus noise. We could put one Gaussian around the end pose, but in practice a separate noise on each of the three parts matches real robots better. The robot makes three separate motions, and each goes wrong in its own way: a wrong first turn sends it off at a wrong angle, a wrong drive changes only the distance, a wrong second turn changes only the end heading. Section 5.2 shows the shapes this gives. Each noise has a standard deviation that grows with the size of the motion, because a longer drive or a bigger turn gives the wheels more chance to slip (Freiburg slides):

$$\sigma_{\text{rot1}} = \alpha_1 |\delta_{\text{rot1}}| + \alpha_2 \, \delta_{\text{trans}}$$

$$\sigma_{\text{trans}} = \alpha_3 \, \delta_{\text{trans}} + \alpha_4 (|\delta_{\text{rot1}}| + |\delta_{\text{rot2}}|)$$

$$\sigma_{\text{rot2}} = \alpha_1 |\delta_{\text{rot2}}| + \alpha_2 \, \delta_{\text{trans}}$$

The four numbers α_1 to α_4 are set for each robot:

- $\alpha_1 = 0.1$: turning error per radian turned;
- $\alpha_2 = 0.1$ rad per m: turning error per metre driven;
- $\alpha_3 = 0.1$: driving error per metre driven;
- $\alpha_4 = 0.01$ m per rad: driving error per radian turned.

For our step:

$$\sigma_{\text{rot1}} = 0.1 \times 0.5 + 0.1 \times 0.479$$

$$= 0.098 \text{ rad}$$

$$\sigma_{\text{trans}} = 0.1 \times 0.479 + 0.01 \times 1.0$$

$$= 0.058 \text{ m}$$

$$\sigma_{\text{rot2}} = 0.098 \text{ rad}$$

So the first turn is 0.500 rad give or take about 0.1 rad (6°), and the drive 0.479 m give or take about 6 cm. With these three standard deviations the model can score any end pose.

4.4 Scoring a possible end pose

The question a grid filter asks: how likely is it that the robot, starting at (2, 1, 0.524), really ended at the pose (2.27, 1.38, 1.50)? The odometry model answers in three steps (Freiburg slides, `motion_model_odometry`):

1. Work out the turn, drive and turn that would take the start pose to this pose, with the formulas of Section 4.2. We mark them with hats; for this pose they are $\hat{\delta}_{\text{rot1}} = 0.429$, $\hat{\delta}_{\text{trans}} = 0.466$ and $\hat{\delta}_{\text{rot2}} = 0.547$.
2. Compare each with what odometry reported, using the normal density with the standard deviations of Section 4.3.
3. Multiply the three densities, because the three noises are taken as independent (G-934).

The three comparisons:

$$p_1 = \mathcal{N}(0.500 - 0.429;\ 0.098) = 3.14$$

$$p_2 = \mathcal{N}(0.479 - 0.466;\ 0.058) = 6.72$$

$$p_3 = \mathcal{N}(0.500 - 0.547;\ 0.098) = 3.63$$

Here $\mathcal{N}(a;\ \sigma)$ is the normal density at a distance a from the centre, with standard deviation σ:

$$\mathcal{N}(a;\ \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-a^2/(2\sigma^2)}$$

The density of the pose is the product:

$$3.14 \times 6.72 \times 3.63 = 76.7$$

The value 76.7 is a density per radian, per metre and per radian, so it can be far above 1. On its own it means little; it is useful for comparison. The pose (2.10, 1.60, 1.50), which needs a first turn of 0.882 rad instead of 0.5, scores 8.7×10^{-7}: about a hundred million times less likely. The notebook `RO-008-probabilistic-motion-models.ipynb` computes both.

Scoring every pose on a fine grid around the start and shading each by its density draws the shape of the distribution. Its typical shape is curved like a banana, a **banana-shaped distribution** (G-2328). Section 5.2 shows why.

5. Drawing sample poses

> **Key point:** Particle filters need random end poses that follow the motion model. We get them by drawing a random noise for each of the three parts and driving turn, drive, turn.

5.1 Why sample instead of scoring

The cloud of Figure 1 is a set of 300 sampled end poses; this section draws such a cloud. Scoring needs a grid over all poses, and a pose has three numbers, so the grid is large: 100 values each of x, y and θ make a million cells to score after every move. A cloud of sample poses describes the same distribution where it matters: more samples where the density is high, few where it is low. To build the cloud we need to draw random numbers that follow a chosen distribution. Drawing a random value so that, over many draws, values come up as often as the distribution says is **sampling** (G-2326) from that distribution.

5.2 Sampling the odometry model

To draw one end pose (Freiburg slides, `sample_motion_model`):

1. Draw a noisy first turn, drive and second turn: each odometry value plus a normal sample with the standard deviation of Section 4.3.
2. Turn by the noisy first turn and drive the noisy distance:

$$x' = x + \hat{\delta}_{\text{trans}} \cos(\theta + \hat{\delta}_{\text{rot1}})$$

$$y' = y + \hat{\delta}_{\text{trans}} \sin(\theta + \hat{\delta}_{\text{rot1}})$$

3. Turn by both noisy turns:

$$\theta' = \theta + \hat{\delta}_{\text{rot1}} + \hat{\delta}_{\text{rot2}}$$

Worked sample. Suppose the three draws come out as $+0.05$ rad, -0.03 m and -0.02 rad:

$$\hat{\delta}_{\text{rot1}} = 0.500 + 0.05 = 0.550$$

$$\hat{\delta}_{\text{trans}} = 0.479 - 0.03 = 0.449$$

$$\hat{\delta}_{\text{rot2}} = 0.500 - 0.02 = 0.480$$

The robot drives in the direction:

$$0.524 + 0.550 = 1.074 \text{ rad}$$

$$x' = 2 + 0.449 \times 0.477 = 2.214$$

$$y' = 1 + 0.449 \times 0.879 = 1.395$$

$$\theta' = 0.524 + 0.550 + 0.480 = 1.554$$

This sample ends 3.5 cm left of and 1.4 cm below the model's end point, and faces 1.7° further left.

Figure 6 draws 500 such samples for three noise settings. With the Note's noise the cloud is round. When turning noise dominates, the samples swing along a circle around the start, because a wrong first turn sends the robot off at a wrong angle but the same distance: that is the banana. When driving noise dominates, they spread along the line of travel. All of this needs odometry; a robot without encoders has only its command.

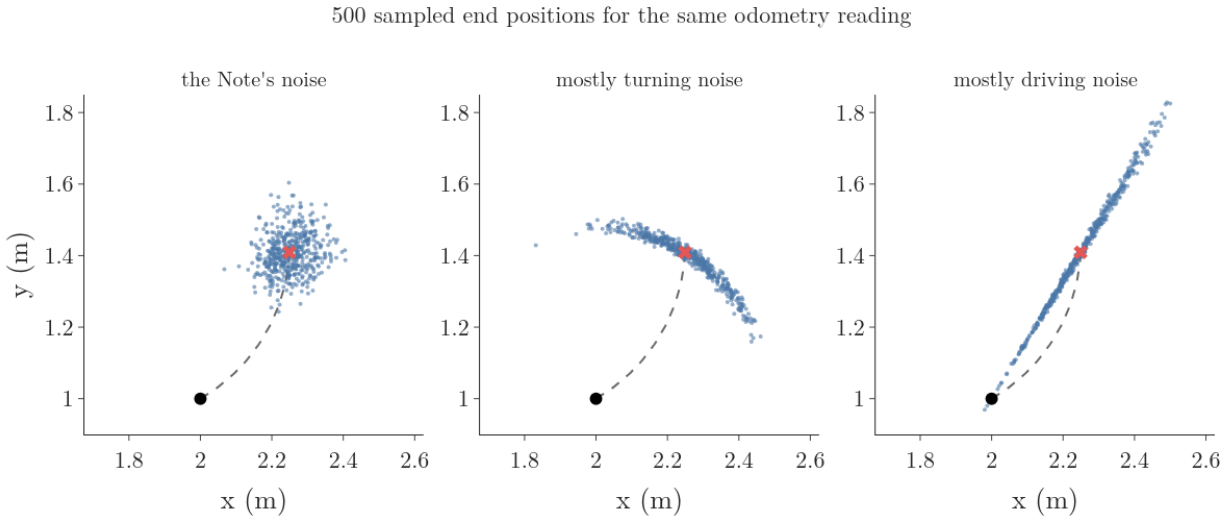


Figure 6: 500 sampled end positions for the same odometry reading under three noise settings: the Note's noise gives a round cloud around (2.249, 1.409); mostly turning noise bends it into a banana around the start; mostly driving noise stretches it along the line from the start

6. The velocity motion model

> **Key point:** When only the commanded speeds are known, the robot is taken to drive an arc of a circle. Noise on the commanded linear and angular velocity, plus an extra noise on the final heading, gives the distribution of end poses.

6.1 The exact arc from the commanded speeds

A robot without encoders knows only the command of Figure 1. The velocity model takes the control to be the commanded linear velocity v (how fast the robot drives forward) and angular velocity ω (how fast its heading turns) (Freiburg slides):

$$u_t = (v,\ \omega) = (0.5 \text{ m/s},\ 1 \text{ rad/s})$$

held for a time step $\Delta t = 1$ s. RO-001 predicted the path with small Euler steps (G-2299). With speeds held fixed we can do better and compute the end pose exactly, because the robot then drives an arc of a circle about one fixed point.

The centre of the arc. A robot driving at v while turning at ω circles one fixed point, the **instantaneous centre of curvature** (ICC) (G-2529), at the turning radius (G-2293) on its left, square to its heading (RO-001):

$$R = \frac{v}{\omega} = \frac{0.5}{1} = 0.5 \text{ m}$$

Square to the heading, to the left, is the direction $(-\sin\theta,\ \cos\theta)$, the robot's left arrow in RO-001. So the ICC is:

$$x_c = x - R\sin\theta$$

$$y_c = y + R\cos\theta$$

At $\theta = 0$ this gives the point $(x,\ y + R)$, straight to the left of a robot facing along x, as it should. For our robot:

$$x_c = 2 - 0.5 \times 0.5 = 1.750$$

$$y_c = 1 + 0.5 \times 0.866 = 1.433$$

The end pose. In Δt the robot turns by $\omega\Delta t$ about the ICC, so its heading becomes $\theta + \omega\Delta t$, and it sits on the circle on the right of its new heading:

$$x' = x_c + R\sin(\theta + \omega\Delta t)$$

$$y' = y_c - R\cos(\theta + \omega\Delta t)$$

$$\theta' = \theta + \omega\Delta t$$

At $\Delta t = 0$ these give back the start pose, as they should. Our robot turns by 1 rad to the heading 1.524 rad (87.3°):

$$x' = 1.750 + 0.5 \times 0.999 = 2.249$$

$$y' = 1.433 - 0.5 \times 0.047 = 1.409$$

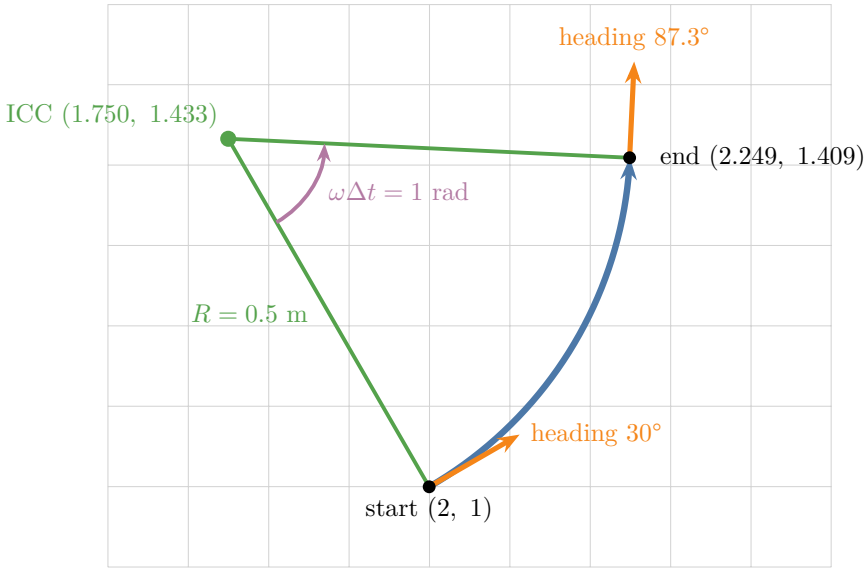


Figure 7: The velocity model without noise: v = 0.5 m/s and omega = 1 rad/s give a turning radius of 0.5 m; the ICC (1.750, 1.433) lies 0.5 m to the robot's left; in 1 s the robot turns 1 rad about it and ends at (2.249, 1.409) facing 87.3 degrees

Figure 7 draws the construction. One Euler step of 1 s would instead put the robot at (2.433, 1.250), 24 cm away, because it drives straight along the start heading.

A gotcha: $R = v/\omega$ has no value when $\omega = 0$. A robot driving straight has no ICC, so code must use the straight-line step when ω is zero or tiny:

$$x' = x + v\Delta t \cos\theta$$

$$y' = y + v\Delta t \sin\theta$$

6.2 Noise, and why it needs a third number

The noise goes on the two commanded speeds, again growing with their size (Freiburg slides):

$$\hat{v} = v + \text{sample}(\alpha_1|v| + \alpha_2|\omega|)$$

$$\hat{\omega} = \omega + \text{sample}(\alpha_3|v| + \alpha_4|\omega|)$$

where sample(b) draws normal noise with standard deviation b. With $\alpha_1 = 0.1$, $\alpha_2 = 0.02$, $\alpha_3 = 0.1$ and $\alpha_4 = 0.1$, the linear velocity varies by about 0.07 m/s and the angular velocity by about 0.15 rad/s.

These two noises are not enough. A pose has three numbers, but $\hat{v}$ and $\hat{\omega}$ are only two. For every end position, exactly one circular arc leaves the start along the start heading and reaches it, so the end heading is fixed by the end position (Figure 8, left). A real robot can end at the same spot facing a little differently. The model therefore adds a third noise, $\hat{\gamma}$, a small extra turn at the end (Freiburg slides):

$$\hat{\gamma} = \text{sample}(\alpha_5|v| + \alpha_6|\omega|)$$

$$\theta' = \theta + \hat{\omega}\Delta t + \hat{\gamma}\Delta t$$

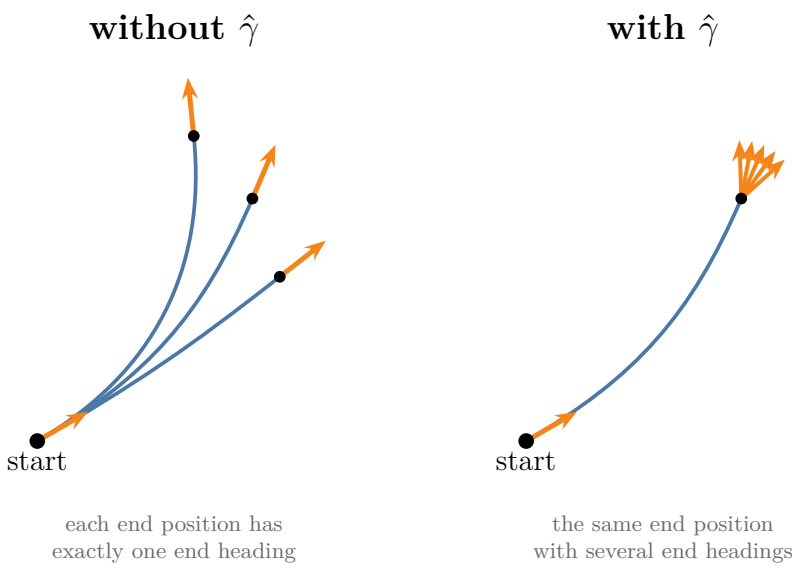


Figure 8: Left, without gamma: three arcs leave the start along its heading, and each end position has exactly one end heading. Right, with gamma: the same end position can be reached with several end headings

The notebook checks this with 2 million samples. Among the samples that end within 2 mm of (2.249, 1.409), the end headings have a standard deviation of only 0.24° without $\hat{\gamma}$, and of 1.71° with it ($\alpha_5 = \alpha_6 = 0.02$).

Worked sample. Suppose the draws come out as $\hat{v} = 0.54$ m/s, $\hat{\omega} = 0.9$ rad/s and $\hat{\gamma} = 0.02$ rad/s:

$$R = 0.54/0.9 = 0.6 \text{ m}$$

$$x_c = 2 - 0.6 \times 0.5 = 1.700$$

$$y_c = 1 + 0.6 \times 0.866 = 1.520$$

The heading after the arc:

$$0.524 + 0.9 \times 1 = 1.424 \text{ rad}$$

$$x' = 1.700 + 0.6 \times 0.989 = 2.294$$

$$y' = 1.520 - 0.6 \times 0.146 = 1.432$$

$$\theta' = 1.424 + 0.02 \times 1 = 1.444$$

A faster, gentler drive: the sample ends 4.4 cm further right and 2.2 cm higher than the noise-free end, facing 4.6° less far round. Neither model knows the room, so either can still put a sample inside a wall.

7. Ruling out poses inside walls

> **Key point:** A map tells the robot where it cannot be. For small steps we multiply the motion model by "is this pose free?"; when sampling, we throw away and redraw any sample that lands inside an obstacle.

The cloud of Figure 1 can still hold poses where the robot cannot be: the motion models so far know nothing about the surroundings, so they put samples inside walls (Figure 9, left, red). With a map m of the free space we want the motion model that also knows the map:

$$p(x_t \mid u_t,\ x_{t-1},\ m)$$

Computing it exactly is hard, because it asks whether the robot could have reached x_t without hitting anything along the way. For small steps there is an easy approximation (Freiburg slides, 2012 edition; Berry, Lecture 3-2b):

$$p(x_t \mid u_t, x_{t-1}, m)$$

$$\approx \eta\, p(x_t \mid u_t, x_{t-1})\, \pi(x_t)$$

- $\pi(x_t)$ is 1 if the pose x_t is free in the map and 0 if it is inside an obstacle;
- η is a number that rescales the result so that it adds up to 1 again.

This is the **map-consistent motion model** (G-2329). When sampling, it becomes a simple rule: draw a sample with the motion model; if it lies inside an obstacle, throw it away and draw again. Keeping only the draws that pass a test, and drawing again otherwise, is **rejection sampling** (G-2330) (Freiburg slides).

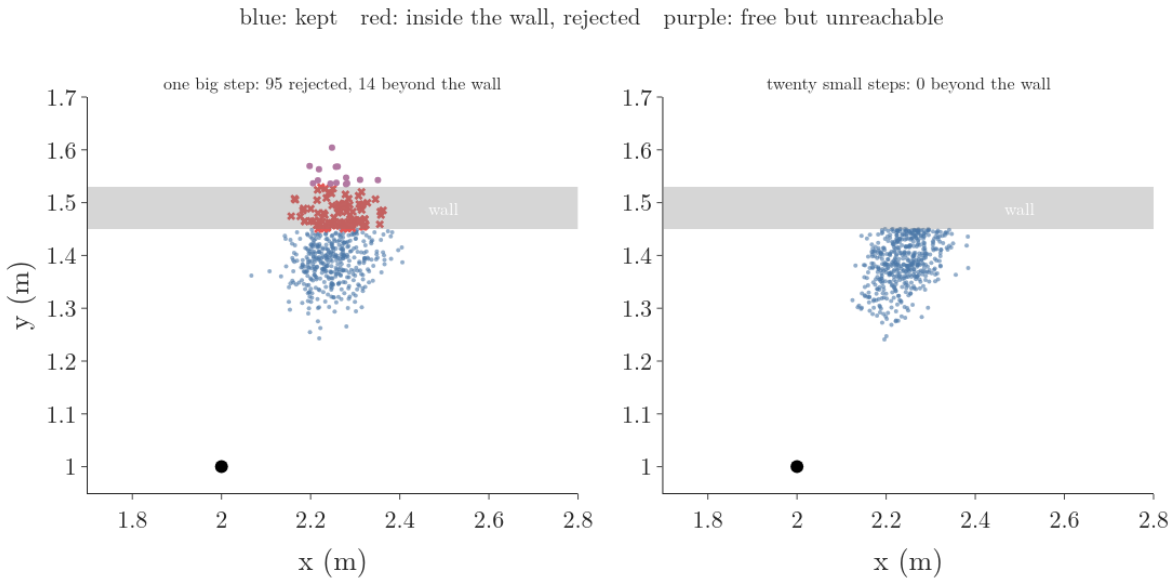


Figure 9: Left: one big step of 0.48 m; of 500 samples, 95 land inside a wall and are rejected, but 14 land beyond the wall in free space and are kept, though the robot could not have driven through the wall. Right: the same motion in twenty small steps, rejecting and redrawing at each step: no sample gets beyond the wall

The approximation checks only where a sample ends, not the path to it. That is why it needs small steps. In Figure 9, left, a wall 8 cm thick lies across the cloud of our 0.48 m step. Of 500 samples, 95 end inside the wall and are rejected, but 14 end beyond it, in free space, and are kept, although the robot could never have driven through the wall. On the right, the same motion is made in twenty small steps of about 2.4 cm, each with the noise raised so that the final cloud has the same standard deviation, and every sample that lands in the wall is redrawn. A small step cannot cross the wall in one jump, so no sample gets beyond it. A common rule of thumb is to keep each step below half the robot's diameter.

8. Summary

Model	Control u_t	Noise on	Use when
Odometry model	odometry's pose change: turn, drive, turn	each of the three parts	encoders exist; filtering after the move
Velocity model	commanded v, ω for Δt	v, ω and an extra end turn γ	no encoders, or predicting before the move (planning)
Map-consistent model	either of the above, plus a map	as above; poses inside obstacles get 0	a map is known and steps are small

- The command of the opening, 0.5 m/s and 1 rad/s for 1 s, does not lead to the one pose (2.249, 1.409) but to a cloud around it, because wheels slip and motors lag; a motion model $p(x_t \mid u_t, x_{t-1})$ describes that cloud; its values are densities, so only areas are probabilities (0.048 for the band 1.09 to 1.11 m) and a density can exceed 1.
- Wheel odometry turns encoder ticks into wheel distances, the robot's move and a new pose; it drifts because each error stays in the running sum, and a 1 percent wrong wheel radius already puts the robot 2.49 m off after 10 m, so robots need calibration.
- The odometry model splits each reported move into turn, drive, turn and puts noise on each, because that matches real robots better than one Gaussian on the end pose; its standard deviations grow with the motion, because bigger motions slip more.
- Scoring a pose multiplies three normal densities (76.7 for a close pose, 8.7×10^{-7} for a far one), so filters can compare poses on a grid.
- Sampling draws noisy turn, drive, turn and moves the robot.
- Rotation noise bends the cloud into a banana, because a wrong turn sends the robot the same distance in a wrong direction; with no readings the cloud keeps growing.
- The velocity model drives an exact arc about the ICC, $R = v/\omega$ to the robot's left; it needs a third noise γ, because two noisy speeds alone tie the end heading to the end position.
- A map rules out poses inside obstacles by rejecting samples there, which is safe only for small steps, because the check looks at where a sample ends, not at the path to it.

The robot can now say how far a move blurs its pose, score any end pose and draw the cloud. Still missing: a reading only means something against a map of where things are, which is maps and landmarks.

9. Sources

Built from

- Cyrill Stachniss channel (University of Bonn), "Motion Models", YouTube, https://www.youtube.com/watch?v=IVTV7vJgIkU (Stachniss channel, Motion models)

- Carlotta A. Berry, "Advanced Mobile Robotics: Lecture 3-2b Probabilistic Motion Models", YouTube, https://www.youtube.com/watch?v=sTQGT0ar6-g (Berry, Lecture 3-2b)
- Duckietown, "12 - Odometry", YouTube, https://www.youtube.com/watch?v=154DmLoGWic (Duckietown)
- NPTEL IIT Madras, Introduction to Robotics, "#38 Odometry Motion Model", YouTube, https://www.youtube.com/watch?v=PPwih9x12YQ (NPTEL #38)
- Büscher, D. (2023). "Probabilistic Motion Models", *Introduction to Mobile Robotics* slides, University of Freiburg (companion material to Thrun et al. 2005 ch.5): odometry and velocity models, noise, sampling normal and triangular noise, rejection sampling, map-consistent motion. http://ais.informatik.uni-freiburg.de/teaching/ss23/robotics/slides/06-motion-models.pdf (Freiburg slides)
- Burgard, W., Stachniss, C., Bennewitz, M. and Arras, K. (2012). "Probabilistic Motion Models", *Introduction to Mobile Robotics* slides, University of Freiburg, http://ais.informatik.uni-freiburg.de/teaching/ss12/robotics/slides/06-motion-models.pdf (Freiburg slides, 2012 edition): the map-consistent approximation.

Other references

- Thrun, S., Burgard, W. and Fox, D. (2005). *Probabilistic Robotics*. MIT Press. §5.3 velocity motion model, §5.4 odometry motion model, §5.5 motion and maps. Cited only where the Freiburg slides confirm it.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Motion model $p(x_t \mid u_t, x_{t-1})$ (G-2319)	The probability distribution of a robot's pose after a move, given its pose before and the control (command or odometry); it describes how uncertain motion is, so a filter can score possible end poses or draw them at random.
Wheel encoder (G-2320)	A sensor that counts how far a wheel turns, for example a slotted disc on the shaft that gives one tick per slot passing a light sensor; its counts are the raw data of wheel odometry.
Wheel odometry (G-2321)	Working out a robot's pose by adding up the wheel motion its encoders measure: ticks to wheel distances, to the robot's move and turn, to a new pose; accurate over short distances but it drifts, because every error stays in the sum.

Term	Meaning
Odometry calibration (G-2322)	Measuring one robot's real wheel radii and wheel separation, for example by driving known paths, so that odometry has no systematic error; a 1 percent radius error alone puts a robot 2.49 m off after 10 m.
Odometry motion model (G-2323)	A motion model that splits the pose change reported by odometry into a first turn, a straight drive and a second turn, and adds independent noise to each, with a standard deviation that grows with the motion; used when wheel encoders exist.
Velocity motion model (G-2324)	A motion model that takes the commanded linear velocity v and angular velocity ω, moves the robot along the exact arc about the ICC that they give, and adds noise to both speeds plus an extra final turn; used without encoders or to predict before a move.
atan2 (G-2325)	A function atan2(rise, run) that returns the direction of a line over the full circle from its rise and run given separately; needed because the plain inverse tangent of the ratio cannot tell opposite directions apart.
Banana-shaped distribution (G-2328)	The curved cloud of end positions a motion model gives when turning noise is large: a wrong first turn sends the robot the right distance in a wrong direction, so the positions spread along an arc around the start.
Sampling (from a distribution) (G-2326)	Drawing random values so that, over many draws, each value comes up as often as a given distribution says; it turns a motion model into a cloud of possible poses for particle filters.
Map-consistent motion model (G-2329)	A motion model that also uses the map, approximated for small steps by multiplying the plain motion model by 1 for free poses and 0 for poses inside obstacles; it stops the robot's estimate from passing through walls.
Rejection sampling (G-2330)	Drawing samples and keeping only those that pass a test, drawing again otherwise; used to keep motion samples out of obstacles.

Term	Meaning
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Linear velocity v (G-2291)	How fast a robot's reference point (for a differential drive, the middle of the axle) moves along its heading, in m/s; for a differential drive, the average of the two wheel speeds, so it sets how fast the robot advances.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.
Dead reckoning (G-2311)	Working out a robot's position from a known start by adding up its own moves, without looking at its surroundings; simple, but its error grows with every move, so it needs readings to correct it.
Standard deviation of a random variable (G-1870)	The square root of a random variable's variance, in the units of X; it says how far values typically fall from the expected value.
Gaussian distribution (G-827)	Another name for the normal distribution: the symmetric, bell-shaped continuous distribution set by its mean and standard deviation, used to model many measurements.
Probability density (G-1569)	The height of a continuous distribution's curve; compares how likely nearby values are.
Rolling without slipping (G-2290)	A wheel moving so that its contact point does not skid: it moves forward by its radius times the angle it turns and never slides sideways, which is what lets wheel turns predict motion.
Forward kinematics (wheeled robot) (G-2294)	Working out the robot's linear and angular velocity from its wheel speeds, used to predict where the robot goes.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.

Term	Meaning
Independent events (G-934)	Events where one happening does not change the probability of the other.
Instantaneous centre of curvature (ICC) (G-2529)	The point a turning robot's whole body circles at a given moment; for a differential drive it lies on the axle line at distance $R = v/\omega$ from the midpoint, so it fixes how fast each wheel must move.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.